invoke one function with an initial value and then invoke a second function with the result from the first function:

```
initialValue = 5
intermediateValue = firstFunction initialValue
finalValue = secondFunction intermediateValue
```

Why is the intermediate value there? It doesn't make the program any clearer. Remove it:

```
initialValue = 5
secondFunction (firstFunction initialValue)
```

The same applies to calculating the tax. Instead of doing each calculation individually,

```
netProfitForProducts = netProfit products
taxForProducts = tax netProfitForProducts
```

you can put them together:

```
taxForProducts = tax (netProfit products)
```

So far, nothing earth-shattering. Think for a second, though; instead of assigning the result to a variable, what does a function that evaluates the final value look like?

```
taxForProducts = (products) -> tax (netProfit products)
```

You do this often, so you should get to know it properly by learning what it's called. It's called `compose`:

```
compose = (f, g) -> (x) -> f g x
```

This looks scary, but it's the classic way to express compose. You'll get used to it, one day.

This little `compose` function lets you join together any two functions:

```
taxForProducts = compose tax, netProfit
loyaltyDiscountForUser = compose loyaltyDiscount, userSpend
```